IBC 모듈 설정 분석 문서

개요

이 문서는 Mars 블록체인 애플리케이션의 IBC (Inter-Blockchain Communication) 모듈 설정과 초기화 과정을 상세히 분석합니다. IBC는 서로 다른 블록체인 간의 안전한 통신을 가능하게 하는 프로토콜로, Mars 체인이 다른 Cosmos 생태계 체인들과 상호 운용할 수 있게 해주는 핵심 기능입니다.

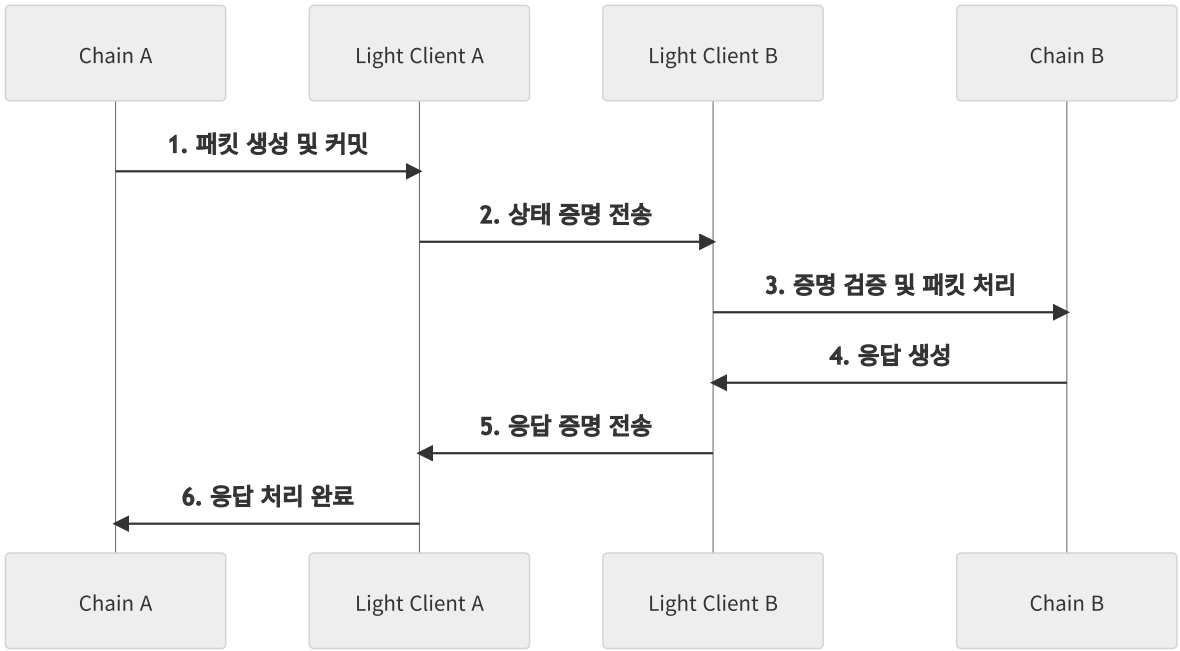
IBC 아키텍처 개요

1. IBC 프로토콜 스택

```
┌─────────────────────────────────────┐
│ IBC 애플리케이션 │
│ (Transfer, ICA Controller/Host) │
├─────────────────────────────────────┤
│ IBC 코어 │
│ (채널, 연결, 클라이언트) │
├─────────────────────────────────────┤
│ 라이트 클라이언트 │
│ (Tendermint, Solo Machine) │
├─────────────────────────────────────┤
│ 합의 레이어 │
│ (CometBFT/Tendermint) │
```

└──┘

2. 크로스체인 통신 흐름



app.go에서의 IBC Keeper 초기화 과정

1. 의존성 주입 시스템에서의 IBC 처리

```
// app.go의 New() 함수에서 IBC 모듈들은 특별한 처리가 필요합니다
func New(...) *App {
// ... 기본 의존성 주입 실행
if err := depinject.Inject(appConfig,
// 표준 Cosmos SDK 모듈들은 자동으로 주입됩니다
```

```
&app.AuthKeeper,

&app.BankKeeper,

// ... 기타 keeper들

// 주의: IBC 관련 keeper들은 여기서 주입되지 않습니다!

// IBC 모듈들은 아직 새로운 의존성 주입 시스템을 완전히 지원하지 않기 때문입니다.

); err != nil {

panic(err)

}

// IBC 모듈들은 별도의 수동 등록 과정이 필요합니다

if err := app.registerIBCModules(appOpts); err != nil {

panic(err)

}

}
```

2. IBC 모듈 수동 등록이 필요한 이유

1. **레거시 호환성**: IBC 모듈들은 Cosmos SDK v0.47 이전의 구조를 따름
2. **복잡한 의존성**: IBC 모듈 간 순환 의존성과 특수한 초기화 순서 필요
3. **동적 라우팅**: 런타임에 결정되는 포트 바인딩과 채널 설정
4. **라이트 클라이언트**: 다양한 합의 알고리즘 지원을 위한 플러그인 구조

IBC 관련 모듈들 상세 분석

1. IBC 코어 (IBCKeeper)

```
// IBC 코어는 모든 IBC 기능의 중앙 관리자입니다
app.IBCKeeper = ibckeeper.NewKeeper(
app.appCodec, // Protocol Buffers 코덱
runtime.NewKVStoreService(app.GetKey(ibcexported.StoreKey)), // IBC 전용 저장소
app.GetSubspace(ibcexported.ModuleName), // 레거시 파라미터 관리
app.UpgradeKeeper, // 체인 업그레이드 처리
govModuleAddr, // 거버넌스 권한 주소
)
```

주요 구성 요소:

- **ClientKeeper**: 라이트 클라이언트 관리 (다른 체인 상태 추적)
- **ConnectionKeeper**: 체인 간 연결 관리 (핸드셰이크, 검증)
- **ChannelKeeper**: 애플리케이션 간 채널 관리 (패킷 라우팅)
- **PortKeeper**: 포트 바인딩 관리 (애플리케이션 식별)

저장되는 데이터:

- 라이트 클라이언트 상태 및 합의 상태
- 연결 정보 (연결 ID, 상대방 체인 정보)
- 채널 정보 (채널 ID, 포트 ID, 상태)
- 패킷 커밋먼트 및 확인 정보

2. IBC Transfer (TransferKeeper)

```
// IBC Transfer는 체인 간 토큰 전송을 담당합니다
```

```
app.TransferKeeper = ibctransferkeeper.NewKeeper(
app.appCodec, // 메시지 직렬화
runtime.NewKVStoreService(app.GetKey(ibctransfertypes.StoreKey)), // Transfer 저
app.GetSubspace(ibctransfertypes.ModuleName), // Transfer 파라미터
app.IBCKeeper.ChannelKeeper, // 패킷 송신 (ICS4Wrapper)
app.IBCKeeper.ChannelKeeper, // 패킷 수신 (포트 키퍼)
app.MsgServiceRouter(), // 메시지 라우팅
app.AuthKeeper, // 계정 관리
app.BankKeeper, // 토큰 전송
govModuleAddr, // 거버넌스 권한
)
```

토큰 전송 메커니즘:

1. **송신 과정:**

```
사용자 → [토큰 에스크로] → 패킷 생성 → 대상 체인 전송
```

2. **수신 과정:**

```
패킷 수신 → 검증 → IBC 토큰(voucher) 발행 → 수신자에게 전송
```

3. **토큰 형태:**

- 원본 체인: `stake` (원본 토큰)
- 대상 체인: `ibc/27394FB092D2ECCD56123C74F36E4C1F926001CEADA9CA97EA622B25F41E5EB2` (IBC 토큰)

저장되는 데이터:

- 토큰 에스크로 정보 (잠긴 토큰 양과 채널 정보)
- Denom Trace (토큰의 원본 체인과 경로 추적)
- 전송 기록 및 타임아웃 정보

3. ICA (Interchain Accounts) 모듈들

3.1 ICA Host (ICAHostKeeper)

```
// ICA Host는 다른 체인으로부터 제어받는 계정을 관리합니다
app.ICAHostKeeper = icahostkeeper.NewKeeper(
app.appCodec, // 메시지 직렬화
runtime.NewKVStoreService(app.GetKey(icahosttypes.StoreKey)), // Host 저장소
app.GetSubspace(icahosttypes.SubModuleName), // Host 파라미터
app.IBCKeeper.ChannelKeeper, // 패킷 송신
app.IBCKeeper.ChannelKeeper, // 패킷 수신
app.AuthKeeper, // ICA 계정 생성
app.MsgServiceRouter(), // 원격 명령 실행
app.GRPCQueryRouter(), // 쿼리 처리
govModuleAddr, // 거버넌스 권한
)
```

Host 기능:

- 원격 체인의 요청으로 ICA 계정 생성
- 원격 명령을 받아 로컬 트랜잭션 실행

- 실행 결과를 원격 체인으로 전송
- 계정 권한 및 실행 가능한 메시지 타입 제한

보안 고려사항:

- `allowMessages` 파라미터로 허용되는 메시지 타입 제한
- 계정 소유권 검증 (올바른 채널을 통한 요청인지 확인)
- 가스 제한 및 실행 시간 제한
- 악의적인 원격 제어 방지

3.2 ICA Controller (ICAControllerKeeper)

```
// ICA Controller는 다른 체인의 계정을 원격으로 제어합니다
app.ICAControllerKeeper = icacontrollerkeeper.NewKeeper(
app.appCodec, // 메시지 직렬화
runtime.NewKVStoreService(app.GetKey(icacontrollertypes.StoreKey)), // Controlle
app.GetSubspace(icacontrollertypes.SubModuleName), // Controller 파라미터
app.IBCKeeper.ChannelKeeper, // 패킷 송신
app.IBCKeeper.ChannelKeeper, // 패킷 수신 (응답 처리)
app.MsgServiceRouter(), // 로컬 메시지 처리
govModuleAddr, // 거버넌스 권한
)
```

Controller 사용 예시:

1. **DeFi 자동화**: 다른 체인의 스테이킹 풀에 자동 위임
2. **크로스체인 거버넌스**: 여러 체인에서 동시 거버넌스 투표

3. **자산 관리**: 멀티체인 포트폴리오 리밸런싱

4. **아비트라지**: 자동화된 크로스체인 차익거래

IBC 라우터 구성

1. 모듈 스택 구성

```
var (
// Transfer 모듈 스택 (v1)
transferStack porttypes.IBCModule = ibctransfer.NewIBCModule(app.TransferKeeper)
// Transfer 모듈 스택 (v2) - 새로운 API 지원
transferStackV2 ibcapi.IBCModule = ibctransferv2.NewIBCModule(app.TransferKeeper
// ICA Controller 스택 - 미들웨어 패턴
icaControllerStack porttypes.IBCModule = icacontroller.NewIBCMiddleware(app.ICAC
// ICA Host 스택 - 독립 모듈
icaHostStack porttypes.IBCModule = icahost.NewIBCModule(app.ICAHostKeeper)
)
```

2. 라우터 등록

```
// IBC v1 라우터 - 기존 프로토콜
ibcRouter := porttypes.NewRouter().
AddRoute(ibctransfertypes.ModuleName, transferStack). // "transfer" 포트
AddRoute(icacontrollertypes.SubModuleName, icaControllerStack). // "icacontrolle
```

```
AddRoute(icahosttypes.SubModuleName, icaHostStack) // "icahost" 포트


// IBC v2 라우터 - 새로운 API
ibcv2Router := ibcapi.NewRouter().
AddRoute(ibctransfertypes.PortID, transferStackV2) // 향상된 Transfer v2


// 라우터를 IBC 키퍼에 설정
app.IBCKeeper.SetRouter(ibcRouter)
app.IBCKeeper.SetRouterV2(ibcv2Router)
```

패킷 라우팅 과정:

1. 패킷 수신 → 포트 ID 추출
2. 라우터에서 포트 ID에 해당하는 모듈 검색
3. 해당 모듈의 패킷 처리 메서드 호출
4. 모듈에서 비즈니스 로직 실행 및 응답 생성

라이트 클라이언트 등록

1. Tendermint 라이트 클라이언트

```
// Tendermint/CometBFT 합의 기반 체인을 위한 클라이언트
tmLightClientModule := ibctm.NewLightClientModule(app.appCodec, storeProvider)
clientKeeper.AddRoute(ibctm.ModuleName, &tmLightClientModule)
```

주요 기능:

- 블록 헤더 검증 (서명, 해시, 검증자 세트)
- 검증자 세트 업데이트 추적
- 머클 증명 검증 (상태 변경 증명)
- 라이트 클라이언트 공격 탐지 및 방어

검증 과정:

1. 새로운 헤더 수신
2. 서명 검증 (2/3+ 검증자 서명 확인)
3. 해시 체인 검증 (이전 헤더와의 연결성)
4. 검증자 세트 변경 검증
5. 상태 업데이트 및 저장

2. Solo Machine 라이트 클라이언트

```
// 단일 서명자 기반 클라이언트
soloLightClientModule := solomachine.NewLightClientModule(app.appCodec, storePro
clientKeeper.AddRoute(solomachine.ModuleName, &soloLightClientModule)
```

특징:

- 단일 개인 키 기반 검증
- 간단한 서명 검증 로직
- 낮은 계산 비용과 빠른 검증
- 개인 지갑이나 IoT 디바이스에 적합

사용 사례:

- 개인 지갑과 체인 간 직접 통신
- IoT 디바이스의 체인 연결
- 간단한 오프체인 서비스 통합

저장소 키 및 파라미터 관리

1. IBC 전용 저장소 키

```
// 각 IBC 모듈은 독립적인 저장소 공간을 가집니다
if err := app.RegisterStores(
storetypes.NewKVStoreKey(ibcexported.StoreKey), // "ibc" - IBC 코어
storetypes.NewKVStoreKey(ibctransfertypes.StoreKey), // "transfer" - 토큰 전송
storetypes.NewKVStoreKey(icahosttypes.StoreKey), // "icahost" - ICA Host
storetypes.NewKVStoreKey(icacontrollertypes.StoreKey), // "icacontroller" - ICA
); err != nil {
return err
}
```

2. 레거시 파라미터 서브스페이스

```
// IBC 모듈들은 아직 collections 기반 파라미터를 사용하지 않습니다
keyTable := ibcclienttypes.ParamKeyTable()
keyTable.RegisterParamSet(&ibcconnectiontypes.Params{})
app.ParamsKeeper.Subspace(ibcexported.ModuleName).WithKeyTable(keyTable)
```

```
app.ParamsKeeper.Subspace(ibctransfertypes.ModuleName).WithKeyTable(ibctransfert
app.ParamsKeeper.Subspace(icacontrollertypes.SubModuleName).WithKeyTable(icacont
app.ParamsKeeper.Subspace(icahosttypes.SubModuleName).WithKeyTable(icahosttypes.
```

파라미터 예시:

- **IBC 코어**: 클라이언트 만료 시간, 연결 타임아웃
- **Transfer**: 전송 활성화 여부, 수신 활성화 여부
- **ICA Controller**: 컨트롤러 활성화 여부
- **ICA Host**: 호스트 활성화 여부, 허용 메시지 타입

모듈 등록 및 라이프사이클

1. 최종 모듈 등록

```
if err := app.RegisterModules(
ibc.NewAppModule(app.IBCKeeper), // IBC 코어
ibctransfer.NewAppModule(app.TransferKeeper), // IBC Transfer
icamodule.NewAppModule(&app.ICAControllerKeeper, &app.ICAHostKeeper), // ICA 통합
ibctm.NewAppModule(tmLightClientModule), // Tendermint 클라이언트
solomachine.NewAppModule(soloLightClientModule), // Solo Machine 클라이언트
); err != nil {
return err
```

```
}
```

2. 모듈 라이프사이클 참여

BeginBlock 단계:

- IBC 코어: 클라이언트 상태 업데이트, 만료된 클라이언트 정리
- Transfer: 타임아웃된 패킷 정리
- ICA: 만료된 채널 정리

EndBlock 단계:

- 패킷 커밋먼트 정리
- 채널 상태 업데이트
- 통계 정보 업데이트

보안 고려사항

1. 권한 관리

```
// 모든 IBC 모듈의 중요한 파라미터 변경은 거버넌스를 통해서만 가능
govModuleAddr, _ := app.AuthKeeper.AddressCodec().BytesToString(
authtypes.NewModuleAddress(govtypes.ModuleName)
)
```

2. 라이트 클라이언트 보안

- **이중 서명 탐지**: 악의적인 검증자의 이중 서명 감지

- **롱 레인지 공격 방어**: 오래된 검증자 세트를 이용한 공격 방지
- **클라이언트 동결**: 공격 탐지 시 클라이언트 자동 동결

3. 패킷 보안

- **패킷 타임아웃**: 일정 시간 후 패킷 자동 만료
- **순서 보장**: 순서가 있는 채널에서 패킷 순서 검증
- **재전송 방지**: 중복 패킷 처리 방지

성능 최적화

1. 배치 처리

- 여러 패킷을 한 번에 처리하여 성능 향상
- 클라이언트 업데이트 배치 처리

2. 캐싱

- 자주 사용되는 클라이언트 상태 캐싱
- 검증 결과 캐싱으로 중복 계산 방지

3. 병렬 처리

- 독립적인 채널 간 병렬 패킷 처리
- 라이트 클라이언트 검증 병렬화

향후 개발 방향

1. App Wiring 지원

IBC v2에서는 완전한 의존성 주입 지원이 계획되어 있어, 수동 등록 과정이 불필요해질 예정입니다.

2. 새로운 라이트 클라이언트

- Ethereum 라이트 클라이언트 (Ethereum과의 브리지)
- Bitcoin 라이트 클라이언트 (Bitcoin과의 연결)
- Polkadot 라이트 클라이언트 (Polkadot 생태계 연결)

3. 고급 IBC 애플리케이션

- 크로스체인 DEX
- 멀티체인 거버넌스
- 크로스체인 스테이킹 파생상품

결론

Mars 체인의 IBC 구현은 Cosmos 생태계의 표준을 따르면서도, 향후 확장성을 고려한 구조로 설계되었습니다. 현재는 수동 등록 방식을 사용하지만, 이는 IBC 모듈들의 복잡성과 레거시 호환성 때문이며, 향후 새로운 App Wiring 시스템이 완전히 지원되면 더욱 간소화될 예정입니다.

IBC를 통해 Mars 체인은 다른 Cosmos 체인들과 안전하고 효율적으로 상호 운용할 수 있으며, 이는 크로스체인 DeFi 생태계 구축의 핵심 기반이 됩니다.